MPA vs SPA | REST vs gRPC

Concepts • Comparison • Project-Based Recommendations

PROJECTS EVALUATED

1. San3ty — a platform for finding and booking verified handworkers.
2. Bride Preparation Website — a planning tool for tracking wedding preparation tasks and purchases.
3. KickZone — a football field discovery and booking platform.

Final team choice: San3ty.

1. What Is the Difference Between MPA and SPA?

Multi-Page Applications (MPAs) and Single-Page Applications (SPAs) are two approaches to building web interfaces. The main difference is how navigation works and how the page content is updated.

Multi-Page Application (MPA)

In an MPA, navigating to another page usually sends a request to the server, which returns a new HTML page. This is common in traditional ASP.NET Core MVC applications using Razor Views.

- Works well for websites with separate pages, forms, content, and administrative workflows.
- Often integrates naturally with server-side rendering, which can help search engines index public content.
- Can be simpler to build and maintain for CRUD-focused applications.
- Navigation may trigger a full-page reload, depending on the implementation.

Single-Page Application (SPA)

In an SPA, the browser loads the main application shell, then JavaScript updates parts of the interface and communicates with APIs without requiring a full page reload for every navigation. Common frameworks include React, Angular, and Vue.

- Well suited to highly interactive interfaces with frequent updates.
- Can provide smooth navigation after the initial load.
- Usually requires more frontend planning, state management, and API integration.
- The initial load and SEO may need extra attention, depending on the rendering strategy.

Quick Comparison

Aspect	MPA	SPA
Navigation	Usually requests a new page from the server	Updates the interface without a full reload
Initial load	Can be relatively simple per page	May load more JavaScript initially
User experience	Traditional and straightforward	Highly interactive after initial loading
SEO	Often straightforward for public content	Depends on rendering and configuration
Complexity	Often simpler for CRUD applications	More frontend state and API complexity
Common stack	ASP.NET Core MVC + Razor Views	React / Angular / Vue + API

How Should We Choose?

There is no universally better option. The decision depends on user needs, interaction level, team experience, project scope, time, and deployment requirements.

Practical rule: choose an MPA for page-based workflows and CRUD systems when simplicity is valuable. Consider an SPA for interfaces that need frequent, rich interactions. A hybrid approach can combine both.

2. Recommended Approach for Our Three Project Ideas

The recommendations below are based on the project descriptions provided. They are practical starting points, not fixed rules; the final choice may change if the requirements grow.

Project 1 — San3ty: Handworkers Service Platform

Project overview: San3ty helps users find service professionals such as plumbers, electricians, and air-conditioning technicians. Users can explore technician profiles, ratings, and service details, then submit a service request. A planned AI feature can help analyze an uploaded issue photo and suggest a likely category.

Recommended approach: MPA with targeted JavaScript/AJAX (a hybrid approach).

- Technician profiles, service details, account pages, and request forms fit naturally into an MPA.
- Photo upload and AI-result display can use JavaScript and an API without requiring the entire application to be an SPA.
- Starting with an MPA reduces frontend complexity and lets the team focus on service-request rules, technician verification, reviews, and booking workflows.
- If the product later needs a highly interactive interface or a separate mobile client, an API can support an SPA in the future.

Why this fits: most core workflows are page- and form-based, with a few interactive features. An MPA with selective client-side enhancements offers a balanced starting point. San3ty is the project the team ultimately selected.

Project 2 — Bride Preparation Website

Project overview: this website helps a bride plan wedding preparations. It can provide categorized checklists, show which items have already been purchased, identify what is still missing, and track progress through the preparation process.

Recommended approach: SPA or hybrid, depending on how interactive the checklist needs to be.

- If users frequently mark items as completed, add or edit purchases, adjust budgets, and see progress update immediately, an SPA can provide a smooth experience.
- If the website mainly provides guidance articles and simple forms, an MPA may be sufficient and easier to maintain.
- A practical middle ground is an MPA for informational pages, with JavaScript updating the checklist and progress indicator without full reloads.

Why this fits: the central feature is tracking what is done and what remains. That benefits from interactive updates, but a full SPA is only justified if the interaction level warrants the added complexity.

Project 3 — KickZone: Football Field Booking Platform

Project overview: KickZone helps users discover football fields, review field details, compare available time slots, and make bookings.

Recommended approach: MPA with JavaScript/AJAX for interactive booking components.

- Search results, field profiles, pricing, and policy pages work well as MPA pages.
- Date selection, available-slot updates, and search-result refreshes can use JavaScript/AJAX.
- This approach keeps the overall application simple while making the booking flow more responsive.

Why this fits: the platform combines information pages with a few interactive booking steps. A hybrid approach can support both without introducing a full SPA too early.

Summary of Project Recommendations

Project	Recommended approach	Main reason

San3ty	MPA + targeted JavaScript/AJAX	Page-based service workflows with selected interactive features
Bride Preparation Website	SPA or hybrid	Interactive checklist, purchase tracking, and progress updates
KickZone	MPA + JavaScript/AJAX	Field information pages plus interactive time-slot booking

3. What Is the Difference Between REST and gRPC?

REST and gRPC are approaches for communication between applications or services over a network. Both can support APIs, but they differ in message formats, service definitions, performance characteristics, and browser integration.

REST

REST is a widely used architectural style for designing APIs around resources and HTTP methods such as GET, POST, PUT, PATCH, and DELETE. REST APIs commonly exchange JSON, which is readable and easy to test.

- Good fit for public APIs, web applications, mobile applications, and integrations.
- Easy to inspect and test with tools such as Postman and curl.
- JSON is human-readable, although messages may be larger than binary formats.
- Actual performance depends on API design, payload size, caching, and implementation.

gRPC

gRPC is a high-performance RPC framework that typically uses HTTP/2 and Protocol Buffers (Protobuf) to define services and encode messages in a compact binary format. Client and server code can be generated from .proto service definitions.

- Useful for internal service-to-service communication where efficient messaging matters.
- Supports unary calls and multiple streaming patterns.
- Binary messages can be smaller and faster to process than JSON in suitable scenarios.
- Direct browser use may require gRPC-Web or a proxy, depending on the client environment.

REST vs gRPC at a Glance

Aspect	REST	gRPC
Common format	Usually JSON	Protocol Buffers (binary)
Transport	Often HTTP/1.1 or HTTP/2	Typically HTTP/2
Service contract	OpenAPI/Swagger can document it	Defined in .proto files
Browser integration	Usually straightforward	May require gRPC-Web or a proxy
Streaming	Possible with suitable mechanisms	Built-in streaming support
Typical fit	Public APIs and web/mobile integration	Internal service-to-service communication

When Should We Choose Each?

- Choose REST when the API needs to be easy for web/mobile clients and external systems to consume.
- Choose gRPC when internal services exchange many messages and need efficient payloads, explicit contracts, or streaming.
- A system can use both: REST for public-facing APIs and gRPC for communication between internal services.

Important: gRPC is not automatically better or faster in every scenario, and REST is not inherently slow. The best choice depends on performance needs, client support, tooling, and integration requirements.

4. Conclusion

MPA is a practical choice for many page-based and administrative workflows. SPA is valuable when the interface needs frequent, rich interaction. A hybrid approach can combine the strengths of both. REST is a strong default for many public APIs, while gRPC is useful for efficient service-to-service communication when its strengths match the requirements.

For the three ideas discussed here, San3ty and KickZone are good candidates for an MPA with targeted JavaScript/AJAX. The Bride Preparation Website may benefit from an SPA or hybrid interface if its checklist and progress tracking need frequent interactive updates.

End of Assignment